ECE551 / EXAM CODE EXAMPLES

Common flip-flops • hold/reset/enable/priority • latch contrast | Q# = supplied practice midterm

1. FF SHELL AND THE TWO ESSENTIAL RULES

For scalar FF fragments use input logic clk,d,rst,rst_n,en,pre_n,S,R,T,J,K and output logic q as needed. For a vector use logic [W−1:0] d,q. **Clocked storage uses <=; combinational next-state/output logic uses =.** Unassigned clocked branches hold; incomplete combinational branches infer a latch.

```
module dff_plain(input logic clk,d,
                 output logic q);
  always_ff @(posedge clk)
    q <= d;
endmodule
```

No reset: q is x until its first captured input in four-state simulation. A reset-free DFF is a valid circuit when startup state is irrelevant.

2. SYNCHRONOUS ACTIVE-HIGH RESET

```
always_ff @(posedge clk)
  if (rst) q <= 1'b0;
  else     q <= d;
```

Reset is checked only at a rising clock edge. Draw a DFF with D = rst ? 0 : d; a library may implement the same behavior with an integrated cell.

3. SYNCHRONOUS ACTIVE-LOW RESET

```
always_ff @(posedge clk)
  if (!rst_n) q <= 1'b0;
  else        q <= d;
```

The suffix _n means active low. It does not make a reset asynchronous; the event control determines that.

4. ASYNCHRONOUS ACTIVE-LOW RESET

```
always_ff @(posedge clk or negedge rst_n)
  if (!rst_n) q <= 1'b0;
  else        q <= d;
```

rst_n falling clears q immediately after NBA, independent of the clock. Draw an active-low clear pin on the FF, not only a synchronous D-input mux.

5. ASYNCHRONOUS ACTIVE-HIGH RESET

```
always_ff @(posedge clk or posedge rst)
  if (rst) q <= 1'b0;
  else     q <= d;
```

6. ASYNCHRONOUS ACTIVE-LOW PRESET

```
always_ff @(posedge clk or negedge pre_n)
  if (!pre_n) q <= 1'b1;
  else        q <= d;
```

Preset initializes to 1, unlike clear to 0. Ex03 and idle-high UART/button synchronizers use this idea.

7. ENABLED FF, RESET HAS PRIORITY

```
always_ff @(posedge clk or negedge rst_n)
  if (!rst_n) q <= 1'b0;
  else if (en) q <= d;
// en=0: hold q; no final else required.
```

Draw D = en ? d : q plus asynchronous clear. For a synchronous reset+enable variant, remove the reset edge from the event control and retain reset as the first condition.

8. A REAL LATCH, FOR COMPARISON [Q2]

```
always_latch
  if (en) q <= d;
```

Latch transparent whenever en=1, including d changes between clock edges. always @(en) misses d changes; use always_latch or always @(*) if explicitly modeling a latch. **A latch is not an edge-triggered FF.**

9. SR FF / STICKY DONE FLAG [Q14,16]

```
always_ff @(posedge clk or negedge rst_n)
  if (!rst_n) q <= 1'b0;
  else if (R) q <= 1'b0;
  else if (S) q <= 1'b1;
```

R/S are synchronous. Both high → R wins; both low → hold. Rename q=done, R=new_request/clear, S=completion to make a sticky flag. Async reset must stay its own if clause: **if(!rst_n), else if(clear), else if(set)**. Combining clear into if(!rst_n || clear) breaks the usual async-reset synthesis template.

10. T FF AND JK FF

```
// T FF: T=1 toggles; T=0 holds.
always_ff @(posedge clk or negedge rst_n)
  if (!rst_n) q <= 1'b0;
  else if (T) q <= ~q;

// JK FF: 00 hold; 01 reset; 10 set; 11 toggle.
always_ff @(posedge clk or negedge rst_n)
  if (!rst_n) q <= 1'b0;
  else case ({J,K})
    2'b00: ;
    2'b01: q <= 1'b0;
    2'b10: q <= 1'b1;
    2'b11: q <= ~q;
    default: q <= 1'bx; // expose unknown controls
  endcase
```

These are separate alternatives. A DFF can implement them with D=q^T or D=(J&~q)|(~K&q). The JK x default makes simulation uncertainty visible; use controls known to be 0/1 in real operation.

11. PULSE VERSUS STICKY FLAG

```
// Registered one-cycle indication for a one-cycle event.
always_ff @(posedge clk or negedge rst_n)
  if (!rst_n) pulse <= 1'b0;
  else        pulse <= event_now;

// Sticky until acknowledged; clear wins.
always_ff @(posedge clk or negedge rst_n)
  if (!rst_n) done <= 1'b0;
  else if (ack) done <= 1'b0;
  else if (event_now) done <= 1'b1;
```

pulse follows sampled event_now; if that condition lasts several cycles, pulse also lasts several cycles. For one pulse per rising event, use an edge detector on page 3.

12. PIPELINE / SWAP / PARTIAL RESET [Q9]

```
// Real two-stage pipeline; both stages reset.
always_ff @(posedge clk or negedge rst_n)
  if (!rst_n) begin q1 <= 0; q2 <= 0; end
  else begin q1 <= d; q2 <= q1; end

// Swap old values, not a pipeline.
always_ff @(posedge clk)
  begin a <= b; b <= a; end

// Practice Q9: q2 is NOT reset.
always_ff @(posedge clk)
  if (!rst_n) q1 <= 1'b0;
  else begin q1 <= d; q2 <= q1; end
```

Q9 equations: D1=rst_n?d:0; D2=rst_n?q1:q2. q2 needs a feedback mux and can remain x after reset. NBA assignments use old q1; blocking q1=d; q2=q1 collapses the intended pipeline in that block.

13. CASEZ LATCH FROM THE PRACTICE [Q2]

```
logic out;
always_comb
  casez ({a,b})
    2'b01: out = ~a;
    2'bz0: out = b;
  endcase
```

Analyze intentionally incomplete code: 00→0,01→1,10→0,11→hold. Draw latch D=b, EN=~(a&b). always_comb may warn. To remove the latch, define the required 11 behavior with an output default or default case; do not invent a hold-free behavior without the specification.

14. COMMON CONTROLLER: START → RUN → FINISH

Complete module. Use this for “start work, wait for finished, output one full-cycle done pulse.” IDLE accepts start; RUN waits; FINISH pulses done, then returns. busy is a Moore output. launch is a Mealy control, so the worker accepts it on the same edge as the controller enters RUN. start must be a pulse; finished must belong to the active operation.

```
module job_controller(
  input  logic clk,rst_n,start,finished,
  output logic launch,busy,done_pulse
);
  typedef enum logic [1:0]
    {IDLE,RUN,FINISH} state_t;
  state_t state,nxt_state;

  always_ff @(posedge clk or negedge rst_n)
    if (!rst_n) state <= IDLE;
    else        state <= nxt_state;

  always_comb begin
    nxt_state = state;
    launch = 0; busy = 0; done_pulse = 0;
    case (state)
      IDLE: if (start) begin
        launch = 1;
        nxt_state = RUN;
      end
      RUN: begin
        busy = 1;
        if (finished) nxt_state = FINISH;
      end
      FINISH: begin
        busy = 1;
        done_pulse = 1;
        nxt_state = IDLE;
      end
      default: nxt_state = IDLE;
    endcase
  end
endmodule
```

Bubble labels: IDLE --start/launch→ RUN; RUN --!finished→ RUN; RUN --finished→ FINISH; FINISH --always→ IDLE. done_pulse=1 throughout FINISH only. State change happens at the edge; combinational outputs then reflect the new state.

15. ATTACH A DATAPATH TO AN FSM [Q8]

Fragments; declare logic [7:0] bit_cnt,sh and logic clr_cnt,inc_cnt,load,shift plus logic [7:0] data. Control signals come from the FSM; stored data updates here, not inside always_comb.

```
always_ff @(posedge clk or negedge rst_n)
  if (!rst_n) bit_cnt <= '0;
  else if (clr_cnt) bit_cnt <= '0;
  else if (inc_cnt) bit_cnt <= bit_cnt + 8'd1;

always_ff @(posedge clk or negedge rst_n)
  if (!rst_n) sh <= '0;
  else if (load) sh <= data;
  else if (shift) sh <= {sh[6:0],1'b0};
```

Do not write bit_cnt=bit_cnt+1 in always_comb. Correct alternatives: generate inc_cnt and clock the update, or compute nxt_cnt from current cnt and assign cnt<=nxt_cnt in a separate FF block. A loop in combinational RTL does not create one increment per clock.

16. HOW TO CUSTOMIZE THIS FSM QUICKLY

For sticky done, replace FINISH's pulse with set_done and add an SR-style register. For a required one-cycle gap between actions, insert a dedicated WAIT state. For a load operation before running, assert load on the accepted IDLE transition. Decode counters/comparators into conditions; default control outputs to 0 and next state to current state. Keep reset/enable groups in separate clocked blocks.

17. PRACTICE Q16: TWO TRANSACTIONS + GAP

Complete **controller module**, to connect to the given SPI master. Assumptions match the exam: strt_cnv is a one-cycle request while idle; chnnl stays stable for command capture; the master accepts wrt on a rising edge and clears done on each new transaction. rd_data is the master's 16-bit output.

```
module a2d_sequence(
  input logic clk,rst_n,strt_cnv,done,
  input logic [2:0] chnnl,
  input logic [15:0] rd_data,
  output logic wrt,cnv_cmplt,
  output logic [15:0] cmd,
  output logic [11:0] res
);
  typedef enum logic [1:0]
    {IDLE,CHNNL,WAIT,RESP} state_t;
  state_t state,nxt_state;
  logic set_cc;

  assign cmd = {2'b00,chnnl,11'b0};
  assign res = rd_data[11:0];

  always_ff @(posedge clk or negedge rst_n)
    if (!rst_n) state <= IDLE;
    else        state <= nxt_state;

  always_comb begin
    nxt_state = state; wrt = 0; set_cc = 0;
    case (state)
      IDLE: if (strt_cnv) begin
        wrt = 1; nxt_state = CHNNL;
      end
      CHNNL: if (done) nxt_state = WAIT;
      WAIT: begin
        wrt = 1; nxt_state = RESP;
      end
      RESP: if (done) begin
        set_cc = 1; nxt_state = IDLE;
      end
      default: nxt_state = IDLE;
    endcase
  end

  always_ff @(posedge clk or negedge rst_n)
    if (!rst_n) cnv_cmplt <= 0;
    else if (strt_cnv) cnv_cmplt <= 0;
    else if (set_cc) cnv_cmplt <= 1;
endmodule
```

The first done edge enters WAIT. The next edge accepts WAIT's wrt and enters RESP: one state-cycle gap. CHNNL and RESP self-loop while done=0. Completion is sticky until a new request. A one-cycle done pulse must not be replaced by an uncleared old completion flag from the worker.

18. INSTANTIATION / WIDTHS FOR THE WRAPPER

Inside an A2D wrapper declare wire [15:0] rd16,cmd; wire done,wrt. Connect the sequencer's rd_data to rd16, and the master's output to rd16. The supplied master module is not in this folder, so these connections use the exam's interface.

```
SPI_mstr16 u_spi(
  .clk(clk), .rst_n(rst_n), .wrt(wrt),
  .cmd(cmd), .done(done), .rd_data(rd16),
  .SS_n(a2d_SS_n), .SCLK(SCLK),
  .MOSI(MOSI), .MISO(MISO)
);
// res is rd16[11:0] through a2d_sequence.
```

Old printed-key fixes: state<=IDLE on reset (not state<=state); blocking nxt_state=state in always_comb; consistent cnv_cmplt spelling; explicit 16-bit read intermediate. Include a reset arrow, both !done self-loops, transition conditions, and wrt/set_cc labels on the bubble diagram.

19. STEERING / TIMED QUALIFICATION FSM

Complete control module. Inputs are comparisons and timer expiry, as in HW3. Timer clear restarts continuous qualification; rider removal outranks imbalance; en_steer is withdrawn immediately when an exit condition occurs. Reset state is OFF.

```
module steer_reference(
  input logic clk,rst_n,tmr_full,
  input logic sum_gt_min,sum_lt_min,
  input logic diff_gt_1_4,diff_gt_15_16,
  output logic clr_tmr,en_steer,rider_off
);
  typedef enum logic [1:0]
    {OFF,BALANCE,ON} state_t;
  state_t state,nxt_state;

  always_ff @(posedge clk or negedge rst_n)
    if (!rst_n) state <= OFF;
    else        state <= nxt_state;

  always_comb begin
    nxt_state = state;
    clr_tmr = 0; en_steer = 0; rider_off = 0;
    case (state)
      OFF: begin
        clr_tmr = 1; rider_off = 1;
        if (sum_gt_min && !sum_lt_min)
          nxt_state = BALANCE;
      end
      BALANCE: begin
        if (sum_lt_min) begin
          rider_off = 1; clr_tmr = 1;
          nxt_state = OFF;
        end else if (diff_gt_1_4)
          clr_tmr = 1;
        else if (tmr_full)
          nxt_state = ON;
      end
      ON: begin
        if (sum_lt_min) begin
          rider_off = 1; clr_tmr = 1;
          nxt_state = OFF;
        end else if (diff_gt_15_16) begin
          clr_tmr = 1; nxt_state = BALANCE;
        end else en_steer = 1;
      end
      default: begin
        nxt_state = OFF;
        clr_tmr = 1; rider_off = 1;
      end
    endcase
  end
endmodule
```

20. TIMER DATAPATH FOR THAT CONTROLLER

Fragment. Choose W so the interval fits. The HW3 diagram assumes the timer always counts unless clr_tmr; the reduction full condition marks its all-ones count. If the spec requires a timer that never rolls over, add the saturation enable shown below.

```
logic [W-1:0] timer;
always_ff @(posedge clk or negedge rst_n)
  if (!rst_n) timer <= '0;
  else if (clr_tmr) timer <= '0;
  else timer <= timer + 1'b1;
assign tmr_full = &timer;
// Saturating variant's final branch:
// else if (!(&timer)) timer <= timer+1'b1;
```

Qualification pattern: OFF clears timer → BALANCE clears on any bad interval → ON only after uninterrupted good time. Restart timer after stepping off. If both weight comparisons are false, retain a previously known rider (hysteresis); do not interpret that region as automatic rider removal.

21. SYNCHRONIZED EDGE DETECTOR [Q12]

Complete module. Two synchronization FFs plus one history FF. IDLE is configurable: 1 for idle-high UART/push button, 0 for idle-low signals. Identical reset stages prevent a reset-generated edge pulse.

```
module sync_edges #(parameter bit IDLE=1'b1)(
  input logic clk,rst_n,async_in,
  output logic rise,fall
);
  logic meta,synced,prev;
  always_ff @(posedge clk or negedge rst_n)
    if (!rst_n) begin
      meta <= IDLE; synced <= IDLE; prev <= IDLE;
    end else begin
      meta <= async_in;
      synced <= meta;
      prev <= synced;
    end
  assign rise = synced & ~prev;
  assign fall = ~synced & prev;
endmodule
```

For an input already synchronous to clk, edge detection alone needs current input and a previous-value FF; extra synchronization stages add latency. Synchronization reduces metastability risk; it does not debounce or safely transfer an arbitrary multibit bus.

22. SLIDING PATTERN MATCH [Q11]

Complete module. MSB is oldest sample; new serial input enters bit0. With target AB, reset zeros do not falsely match before all eight pattern bits arrive. Other patterns may need a valid-sample counter.

```
module pattern_match(
  input logic clk,rst_n,in_stream,
  output logic match
);
  logic [7:0] shft_reg;
  always_ff @(posedge clk or negedge rst_n)
    if (!rst_n) shft_reg <= 8'h00;
    else shft_reg <= {shft_reg[6:0],in_stream};
  assign match = (shft_reg == 8'hAB);
endmodule
```

23. RESET SYNCHRONIZER [EX09]

Complete module. Assert immediately; release after two falling clock edges. The rest of the course design uses rising edges, keeping reset deassertion away from them.

```
module reset_sync(
  input logic clk,RST_n,
  output logic rst_n
);
  logic stage;
  always_ff @(negedge clk or negedge RST_n)
    if (!RST_n) begin stage <= 0; rst_n <= 0; end
    else begin stage <= 1; rst_n <= stage; end
endmodule
```

24. SHIFT / ROTATE / UART DATA REGISTERS

Fragments; use separate appropriately sized registers. Parallel load should outrank shift. These lines manipulate old register bits at the capturing edge.

```
// 8-bit rotate: keeps outgoing bit.
q <= {q[6:0],q[7]};
// 8-bit left shift: outgoing MSB falls off.
q <= {q[6:0],serial_in};
// UART TX 9-bit load, then right shift.
tx_sh <= {tx_data,1'b0};
tx_sh <= {1'b1,tx_sh[8:1]};
// UART RX: 10 samples into a 9-bit register.
rx_sh <= {RX_sync,rx_sh[8:1]};
// final stop is bit8; byte is rx_sh[7:0].
```

UART baud: 5208 clocks/bit → count0..5207; first RX half-period2604 clocks → down-counter2603. Frame=10 bit periods. Busy/done and counters belong in FF blocks; FSM generates load,shift,clear,set.

25. CORRECTED PWM RECOGNITION PROBLEM [Q14]

Complete module. Clear outranks set, allowing duty=0. The registered output uses the pre-edge count; count after the edge has advanced. The missing final else is a FF hold, not a latch.

```
module pwm8(
  input logic clk,rst_n,
  input logic [7:0] mag,
  output logic intensity
);
  logic [7:0] cntr;
  logic set_pwm,clear_pwm;
  always_ff @(posedge clk or negedge rst_n)
    if (!rst_n) cntr <= '0;
    else cntr <= cntr + 8'd1;

  assign set_pwm = ~|cntr;
  assign clear_pwm = (cntr >= mag);
  always_ff @(posedge clk or negedge rst_n)
    if (!rst_n) intensity <= 1'b0;
    else if (clear_pwm) intensity <= 1'b0;
    else if (set_pwm) intensity <= 1'b1;
endmodule
```

For constant mag, high clocks=mag per256-clock period. f_pwm=f_clk/256. mag255 gives255/256, not100%. In the old broken code, synchronous clear was mixed into the async reset test; split it as above.

26. SIGN EXTENSION / CARRY / SATURATION [Q5]

Fragments; vec8 is8 bits, vec16 is16; A/B/S are4, co/cin1. Signed s is16 bits and sat10 is10 bits. Continuous assignments require nets in legacy Verilog; single-driver logic is fine in SV.

```
assign vec16 = SE ? {{8{vec8[7]}},vec8}
                  : {8'h00,vec8};
assign {co,S} = {1'b0,A}+{1'b0,B}+cin;
assign mask = bus8 & 8'h0F;
assign any_low_nibble = |mask;

assign sat10 = (!s[15] && |s[14:9]) ? 10'h1FF :
               (s[15] && ~&s[14:9]) ? 10'h200 :
               s[9:0];
// unsigned u16 ->10: discarded bits nonzero?
assign usat10 = |u16[15:10] ? 10'h3FF : u16[9:0];
```

Saturation tests the new sign bit too. Arithmetic shift requires signed operand: $signed(vec8)>>>2. Mixed signed/unsigned comparisons can be wrong; cast both operands or declare both signed. −65>>>6 gives−2, while signed division −65/64 gives−1.

27. PARAMETERIZED ENABLED COUNTER [Q10]

Complete module. Hold when en=0; optional synchronous clr; natural modulo2^W wrap. terminal indicates all-ones count, not a stopped timer.

```
module counter #(parameter int W=8)(
  input logic clk,rst_n,en,clr,
  output logic [W-1:0] count,
  output logic terminal
);
  always_ff @(posedge clk or negedge rst_n)
    if (!rst_n) count <= '0;
    else if (clr) count <= '0;
    else if (en) count <= count + 1'b1;
  assign terminal = &count;
endmodule
// Instance inside another module:
// counter #(.W(11)) u_cnt(.clk(clk),...);
```

CountN clocks with0..N−1; use ceil(log2N) bits (at least1). A counter representing0..N inclusive needs ceil(log2(N+1)). parameter supports per-instance override; localparam does not. Specify width/sign before adding, multiplying, or comparing.

28. STRUCTURAL / BEHAVIORAL RING [Q3,4]

Alternative simulation models; do not connect both to the same out. EN must begin low long enough to establish out=1. Structural gate outputs are nets; behavioral out is a procedural variable.

```
nand #5 g1(n1,EN,out);
not  #5 g2(n2,n1);
not  #5 g3(out,n2);
// Three inversions; half-period15, period30.

always @(EN or out)
  out <= #15 (EN ? ~out : 1'b1);
```

Blocking out=#15... changes out while busy and misses its own event; delayed NBA returns to waiting before the update, sustaining oscillation. #delays are simulation timing, not synthesis constraints.

29. SIMULATION TRACE / SENSITIVITY [Q6,7]

```
// Race example: a may end0 or1; b=0 at t5.
initial begin a=0; b=1; end
initial begin a=1; #5 b=0; end

// Supplied Q6 form: include intermediate stg1.
always @(a,b,c,en,stg1) begin
  stg1 <= en ? a^b : a|c;
  out1 <= stg1 & c;
end
// Preferred rewrite:
always_comb begin
  stg1 = en ? a^b : a|c;
  out1 = stg1 & c;
end
```

The two sensitivity examples are alternatives. Always_comb's written temporaries should not be relied upon to self-retrigger: compute them in dependency order with blocking assignments.

```
// Executed at time t:
#5 q=d;    // sample & update t+5; block
q=#5 d;    // sample t, update t+5; block
q<=#5 d;   // sample t, NBA t+5; continue now
#5 q<=d;   // wait, sample t+5, NBA t+5
```

30. COMPACT SELF-CHECKING TESTBENCH [Q13]

Standalone testbench with local dataflow DUT model. Replace the assign with a named DUT instance when testing a separate adder. Exhausts512 four-bit A/B/cin combinations. Use an int iterator so it can reach512 without wrapping.

```
`timescale 1ns/1ps
module adder_reference_tb;
  logic [3:0] A,B,S;
  logic cin,co;
  logic [4:0] expected;
  assign {co,S} = {1'b0,A}+{1'b0,B}+cin;
  initial begin
    for (int i=0;i<512;i++) begin
      {A,B,cin} = i[8:0];
      expected = int'(A)+int'(B)+int'(cin);
      #1; // let combinational logic settle
      if ({co,S} !== expected)
        $fatal(1,"adder mismatch at case %0d",i);
    end
    $display("PASS: 512 cases");
    $finish;
  end
endmodule
```

For clocked DUTs: initial clk=0; always #10 clk=~clk; drive inputs on negedge; check @(posedge clk); #1. $display sees current values (often pre-NBA); $strobe sees settled values. $stop pauses interactively; $finish exits batch; $fatal fails. #0 is not a general race fix. For a wait, use a parallel timeout so a missing done cannot hang the test.